

Apostila HTML e CSS – Tópicos Avançados

Semântica em HTML5:

`<header>` = Cabeçalho da página ou seção. Pode haver mais de 1.
`<main>` = Conteúdo principal. **Único por página.**
`<section>` = Agrupa conteúdo relacionado, **sempre com título.**
`<article>` = Conteúdo independente, pode ter header/footer.
`<aside>` = Conteúdo complementar (não principal). Pode estar dentro ou fora do `<main>`. Dentro complementa o conteúdo, fora é algo geral. Tem título
`<footer>` = Rodapé da página ou seção.
`<div>` = Elemento genérico, sem significado semântico. Use só pra layout/estilo, quando outra tag semântica não se encaixa na necessidade.

`<h1>` = Título principal da página (1 por página).
`<h2>` = Seção dentro do `<h1>`.
`<h3>` = Subdivisão do `<h2>`.
(Não pule níveis. Hierarquia é lógica, não estética)

`aria-label` = **Rotulo** invisível, pode ser usado em botões que só tem ícones dentro, ou para explicar um grupo, tipo uma div de contato.

`aria-labelledby` = Linka o texto de um elemento a outro. EX: Se temos um `<h2 id="titulo-form">Formulário de Contato</h2>` e temos um `<form>`, podemos colocar `aria-labelledby="titulo-form"` no formulário. Assim o leitor lê "Formulario de Contato ao entrar no `<form>`

`Role` = Usado em elementos não semânticos como `<div>` e `<span>` para indicar qual seu papel. EX:

```
<div role="navigation">
  <a href="/">Início</a>
  <a href="/sobre">Sobre</a>
</div>
```

`tabindex="0"` = Faz os elementos serem acessíveis pelo tab do notebook.

Meta Tags:

`<meta name="description" content="Resumo da página.">`

<meta name="robots" content="index, follow"> (indexar a páginas e seguir seus links)

<meta property="og:title" content="Título p/ redes sociais"> (exibido ao compartilhar via rede social)

<meta property="og:description" content="Descrição p/ redes sociais"> (Descrição que aparece no compartilhamento)

<meta property="og:image" content="imagemlink"> (imagem exibida no compartilhamento)

Imagens:

alt = Descreve as

loading="lazy" (atrasam o carregamento da imagem até ela estar quase visível na tela)

CSS:

min-width = estilos **a partir** de Xpx. (usado em @media)

max-width = estilos **até** Xpx. (usados em @media)

box-sizing: "border-box"; (Deixa os elementos com o tamanho real, levando em conta borda, padding...)

Unidades Relativas e Absolutas em CSS (px, em, rem):

Unidades Absolutas: px

Usado em bordas, sombras, linhas de grade.... Tudo que envolva tamanho fixo e precisão.

Unidades Relativas:

Em: recebe o tamanho do pai. Se o html tem 16px de font-size, e colocarmos 1.5em no main, o main terá 24px de font-size, e se no section dentro do main colocarmos 2em, vamos ter 48px de font-size (note o efeito cascata, ele pega o tamanho do pai, não do html apenas).

Usado em componentes isolados que precisam escalar com o pai.

Rem: recebe o tamanho do html. Se o html tem 20px e colocamos no main 1.5 rem, o main terá 30px, e se colocarmos 2rem no filho de main, o section, ele terá 40px (sem efeito cascata).

Rem é o mais usado entre px e em, mais abrangente.

Podemos definir um font-size para o html e em :root, criar variáveis com rem. Se o html mudar, a variável muda e o lugar que a usa também.

Tema Escuro:

HTML:

```
<html lang="pt-BR" data-theme="light">  
  <button id="tema-dark"><i class="fa-solid fa-moon"></i></button>  
</html>
```

CSS:

(Quando o atributo data-theme no html mudar de “light” para “dark” o :root será sobrescrito pelos valores de data-theme=“dark”)

```
:root{  
  --background-geral: white;  
  --color-titulos: black;  
}
```

```
[data-theme="dark"]{  
  --background-geral: black;  
  --color-titulos: white;  
}
```

JS:

(colocando valor do localStorage no data-theme assim que o html/js carregar)

```
let tagHtml = document.documentElement;  
let atributoHtml = tagHtml.getAttribute('data-theme');  
tagHtml.setAttribute('data-theme', localStorage.getItem('estado'));
```

(pegamos o botão)

```
const botaoTema = document.getElementById('tema-dark');
```

(adicionamos um evento de click, que dispara uma função)

```
botaoTema.addEventListener('click', () =>{
```

(pegamos o elemento html)

```
let tagHtml = document.documentElement;
```

(pegamos o atributo data-theme do elemento html)

```
let atributoHtml = tagHtml.getAttribute('data-theme');
```

(colocamos no atributo data-theme light se tiver valor dark e dark se tiver valor light)

```
tagHtml.setAttribute('data-theme', atributoHtml === "light" ? "dark" : "light");
```

(mudamos o icone do botao para lua se tiver sol e para sol se tiver lua)

```
botaoTema.innerHTML = atributoHtml === "light"
```

```
? "<i class='fa-solid fa-sun'></i>"
```

```
: "<i class='fa-solid fa-moon'></i>";
```

(armazenamos no localStorage)

```
localStorage.setItem('estado', tagHtml.getAttribute('data-theme'));
```

```
});
```

Campo de E-mail personalizado:

HTML:

```
<div class="input-group">
```

```
  <input type="password" name="password" id="password" placeholder="*****"
  autocomplete="current-password" required>
```

```
  <label for="password">Password</label>
```

```
  <button type="button" class="icon" id="seePassword">
```

```
    <i class="fa-solid fa-eye"></i>
```

```
  </button>
```

```
</div>
```

CSS:

```
.input-group{
  position: relative;
  display: flex;
  align-items: center;
}
```

```
.input-group input{
  padding: 1rem 2.5rem 1rem 1rem;
  border: 2px solid var(--color-texto-secundario);
  outline: none;
  background-color: var(--background-fundo);
  color: var(--texto-input);
}
```

```
.input-group label{
  position: absolute;
  top: -0.6rem;
  left: 0.85rem;
  background-color: var(--background-fundo);
  padding: 0 0.5rem;
```

```

}
.input-group input:focus{border: 2px solid var(--background-verificao-input);}
.input-group input:focus + label {color: var(--background-verificao-input);}
(o + pega o elemento que vem imediatamente depois de input, no caso, label)

.input-group span,button.icon{
  position: absolute;
  right: 0.85rem;
}
.input-group span:hover,button.icon:hover{color: var(--background-verificao-input);}

```

Texto cortando por linha:

CSS:

```

main .divider{
  display: flex;
  align-items: center;
  color: var(--color-texto-secundario);
  text-align: center;
}
.divider::before,      (::before e ::after são elementos fantasmas criados apenas no css)
.divider::after{       (não no html, eles vem antes e depois de um termo no html)
  content: "";
  flex: 1;
  border-bottom: 1px solid var(--color-texto-secundario);
}

```

Visualizar Senha:

HTML:

```

<div class="input-group">
  <input type="password" name="password" id="password"
placeholder="*****" autocomplete="current-password" required>
  <label for="password">Password</label>
  <button type="button" class="icon" id="seePassword">
    <i class="fa-solid fa-eye"></i>
  </button>
</div>

```

JS:

```
const inputSenha = document.getElementById('password');
const iconeSenha = document.getElementById('seePassword');
```

```
iconeSenha.addEventListener('click', () => {
  let verificarTipo = inputSenha.type === "password";

  (se o type do input for password, ele muda pra text e vice-versa)
  inputSenha.type = verificarTipo ? "text" : 'password';

  iconeSenha.innerHTML = verificarTipo
    ? "<i class='fa-solid fa-eye-slash'></i>"
    : "<i class='fa-solid fa-eye'></i>";
});
```

Grid Layout:

Grid é um sistema bidimensional de layout (linhas e colunas).

GRID VS FLEXBOX:

Grid é ideal para macro-layout (cabeçalhos, barras laterais, áreas de conteúdo, rodapés, a “arquitetura” da página). Flexbox é unidimensional (ou linha ou coluna), excelente para micro-layout dentro de componentes (alinhamento em um único eixo: botões numa toolbar, itens dentro de um card).

RESUMO:

Em projetos reais, usa-se Grid para o esqueleto e Flexbox dentro das áreas específicas.

Tornar Grid: `.container {display: grid;}`

Criar Linhas e Colunas:

```
.container {
  display: grid;
  grid-template-columns: 200px 1fr 2fr;  (3 colunas: fixa, 1 fração, 2 frações)
  grid-template-rows: 100px auto;      (2 linhas: fixa e ajustável ao conteúdo)
}
```

(fr (fração): reparte o espaço restante proporcionalmente entre trilhos flexíveis. [Depois de descontar trilhos fixos, o que sobra é dividido por “fatores fr”])

Repetir Colunas ou Linhas: `grid-template-columns: repeat(3, 1fr); (1fr 1fr 1fr)`

Tamanho Mínimo e Máximo:

`grid-template-columns: minmax(180px, 1fr) 2fr;`

(Cria 2 colunas, A primeira: pelo menos 180px, pode crescer até 1 parte do espaço. A segunda: ocupa 2 partes do espaço, se a tela tiver 500px. Suas 3 frações seriam de 166.67px, isso daria 1fr com 166.67 e o 2fr com 333.34, mas... ele tem mínimo de 180px, logo sobra 320px, e o 2fr tem que se satisfazer com isso).

Criar várias linhas automáticas e moldáveis:

`grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));`

Essa linha cria várias colunas automaticamente! A quantidade vai se ajustar conforme o tamanho da tela (ou do container)

(
`repeat(auto-fit, ...)`

Está dizendo: “Repita esse padrão de coluna (Auto-fit) encaixe o máximo de colunas possível, sem quebrar a largura mínima”

`minmax(220px, 1fr)`

Cada coluna criada:

Vai ter no mínimo 220px

Pode crescer até 1fr

)

Isso faz: Se a tela for pequena, menos colunas vão caber(ex: 880, cabem 4 colunas de 220)

Se for grande, mais colunas aparecem (ex: 1000, cabem 4 colunas de 220, sobram 120, 120 não permite mais uma coluna, 120 é dividido nas 4 que tem na tela, cada uma agora tem 250, ou seja 1fr).

Espaçamentos entre linhas e colunas:

```
.container {  
  display: grid;  
  gap: 16px;  
  row-gap: 16px;  
  column-gap: 16px;  
}
```

Se quisermos alinhar os itens do Grid:

`justify-items: center;` (centraliza horizontalmente cada item)

`align-items: center;` (centraliza verticalmente cada item)

Se quisermos alinhar um item específico do Grid: `justify-self` e `align-self`

Posicionamento por Nomes em Grid (Áreas nomeadas):

```
.container {  
  display: grid;  
  grid-template-columns: 160px 1fr 1fr; (3 colunas, 1 de 160px e 2 de 1 fração)  
  grid-template-rows: 80px auto 60px; (3 linhas, de 80px, de 60px, 1 automática)  
  grid-template-areas (isso que define os nomes)  
}
```

EX REAL:

```
grid-template-areas:  
  "header header header"  
  "sidebar content content"  
  "footer footer footer";
```

Isso significa:

Linha 1: header | header | header

Linha 2: sidebar | content | content

Linha 3: footer | footer | footer

"header" ocupa toda a linha 1 (3 colunas).

"sidebar" ocupa a 1ª coluna da linha 2.

"content" ocupa as colunas 2 e 3 da linha 2.

"footer" ocupa toda a linha 3.

Depois pegamos a classe/id de nossos respectivos section/header/footer/aside... e falamos qual fica onde.

```
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.content { grid-area: content; }  
.footer { grid-area: footer; }
```

Regras:

Cada linha do grid-template-areas precisa ter o mesmo número de palavras.

Se você quiser deixar um espaço vazio, usa um ponto “.”

EX de FLEX BOX e GRID LAYOUT:

```
.page {  
  display: grid;  
  grid-template-columns: 240px 1fr;  
  grid-template-rows: 64px 1fr;  
  gap: 24px;  
  grid-template-areas:  
    "sidebar header"  
    "sidebar main";  
}  
.header {  
  grid-area: header;  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 12px;  
}  
.sidebar { grid-area: sidebar; }  
main{ grid-area: main; }
```